FSO System Prototype Based on FPGA



WiComAI Lab Kookmin University

Table Of Content

Table Of Content 2
1. System Overview 3
    1.1 Overview End-to-End Architecture 3
    1.2 Ethernet Packet Encapsulation 6
2. System Architecture Breakdown 7
    2.1 Ethernet Interface TX 7
    2.2 FSO Framer TX 9
    2.2.1 ARQ Controller & Header Generator TX 10
    2.2.2 Payload Generator TX 10
    2.3 Clock Domain Crossing TX (GTH to Logic Clk) 11
    2.4 FEC Encoder TX 12
    2.5 Scrambler Encoder TX 13
    2.6 LPC Encoder TX 14
    2.7 Multiplexer for FSO Framer Encoder TX 14
    2.8 Clock Domain Crossing (Logic clk to GTH) Encoder TX 15
    2.9 GTH 16
    2.10 Demultiplexer for Decoder RX 17
    2.11 Demultiplexer for FEC Decoder RX 18
    2.12 Reassembly for RX 18
    2.13 Ethernet Interface RX 20
3. ARQ Selective Repeat Architecture Breakdown 21
    3.1 Theoretical Capacity and Memory Calculation 21
    3.2 ARQ Protocol Processing Flow 21
    3.3 Hardware Implementation (External DDR4 Integration) 23

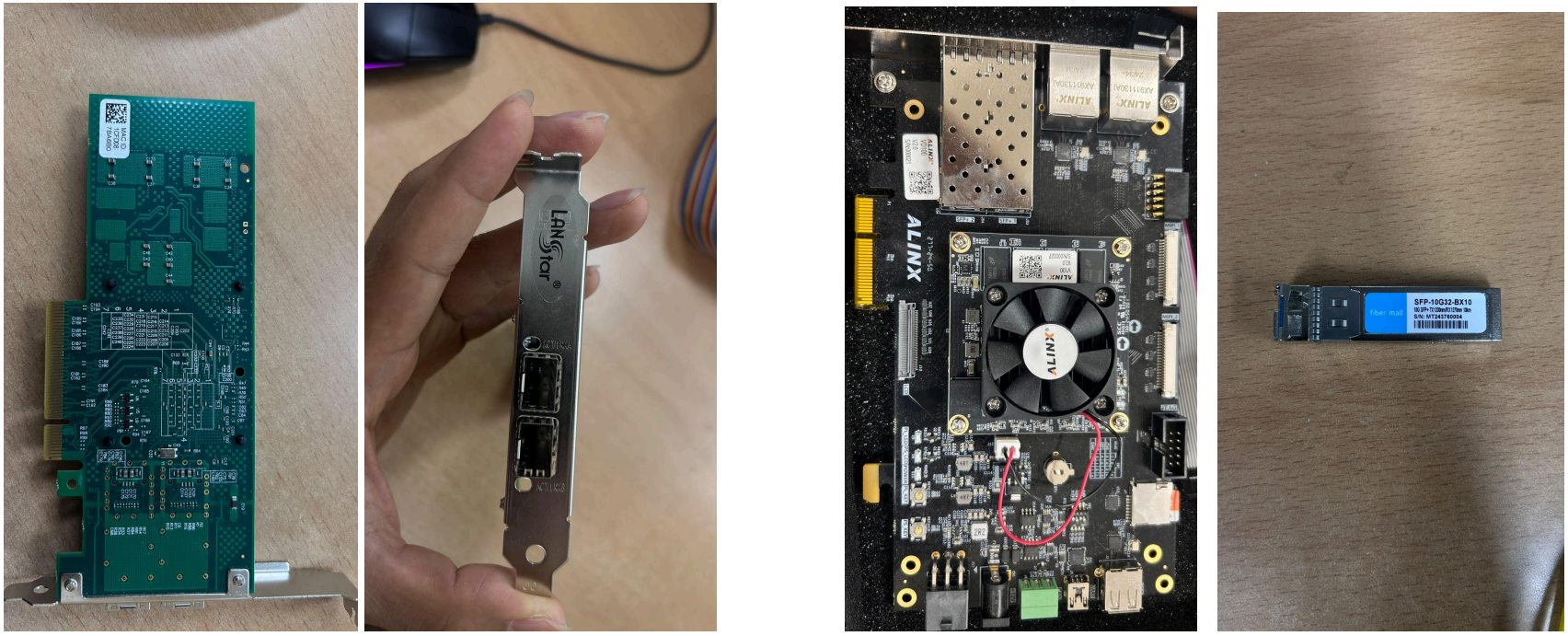
WiComAI Lab
Wireless Communications and Artificial Intelligence Lab

1. System Overview

1.1 Overview End-to-End Architecture

Our system Specification :

- PCIE : NIC (Network) LANstar LS-PCIE-X520-DA2 Intel 82599ES
- FPGA Board : FPGA ALINX AXKU062 Kintex UltraScale (xcku060-ffva1156-2-i)
- SFP+ Port : SFP-10G32-BX10 — 10G SFP+



(a) (b) (c) (d)

Figure 1. (a) PCIE, (b) PCIE SFP+ Port, (c) FPGA Board, (d) SFP+ Port

FPGA TX System Architecture

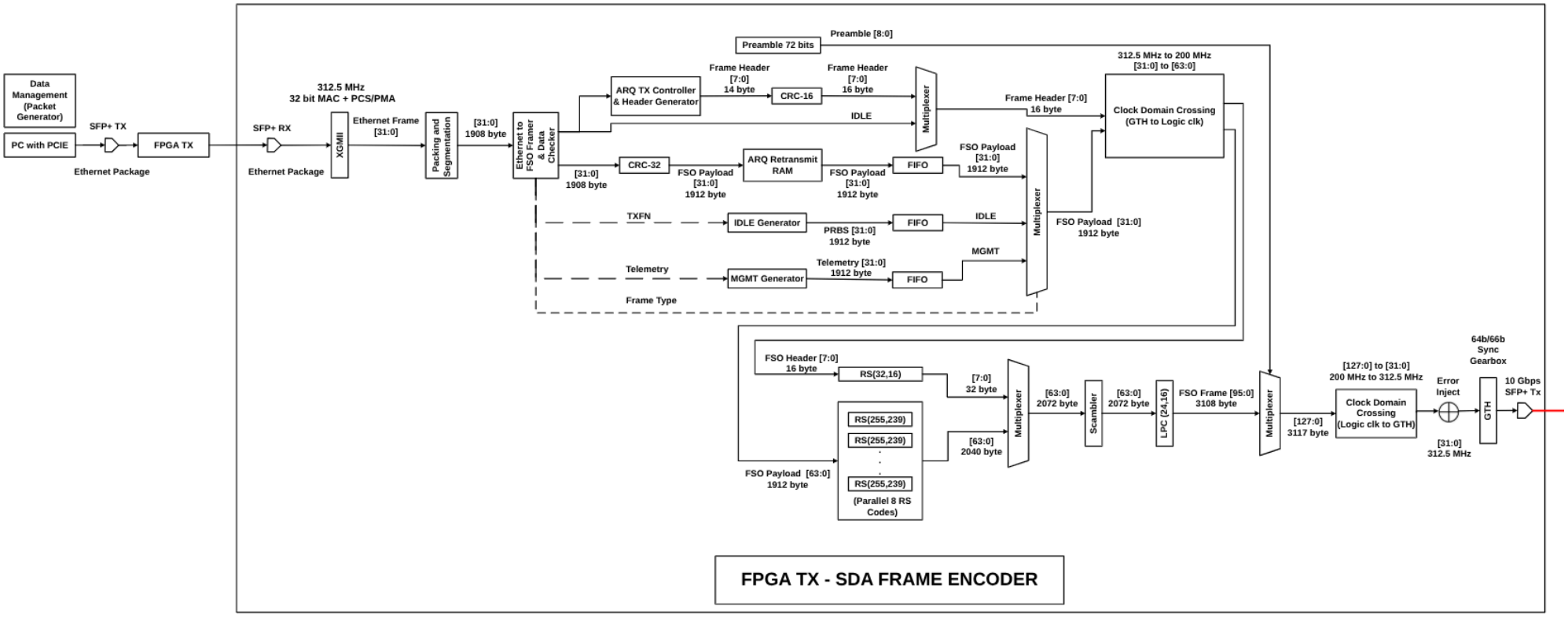


Figure 2. FPGA TX

Flow Structure for FPGA TX :

The data flow originates at PC 1 (packet generator), where the Intel X520 network interface card transmits standard Ethernet packets via its SFP+ TX port into the SFP+ RX port of FPGA 1. Inside FPGA 1, the data undergoes the Encoder process, where it is segmented, appended with ARQ headers, protected by Forward Error Correction (FEC), and passed to the GTH transceiver to be outputted as SDA FSO frames via the SFP+ TX port. This signal travels across the transmission medium and is received by the SFP+ RX port on FPGA 2. Entering through the GTH.

WiComAI Lab
Wireless Communications and Artificial Intelligence Lab

FPGA RX System Architecture

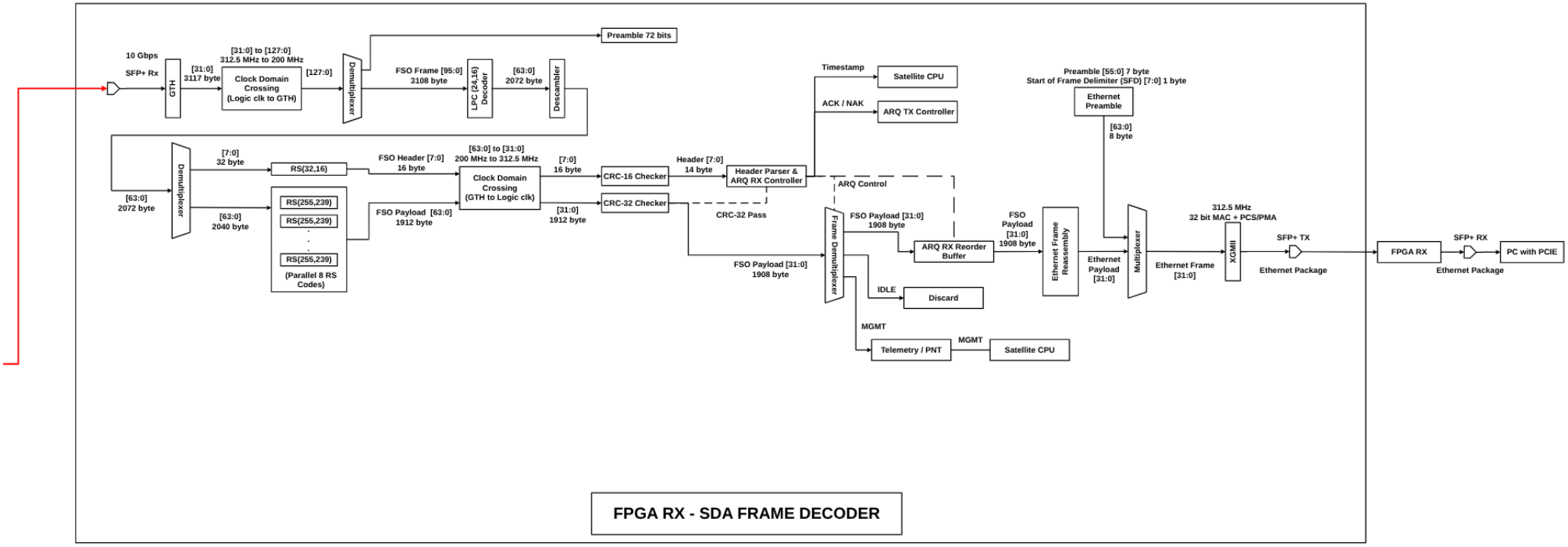


Figure 3. FPGA RX

Flow Structure for FPGA TX :

Entering through the GTH, the data is processed by the Decoder, which corrects transmission errors, reorders the frames, and reassembles them back into standard Ethernet packets. Finally, these intact packets are transmitted out of FPGA 2's SFP+ TX port and received by the Intel X520 card on PC 2.

1.2 Ethernet Packet Encapsulation

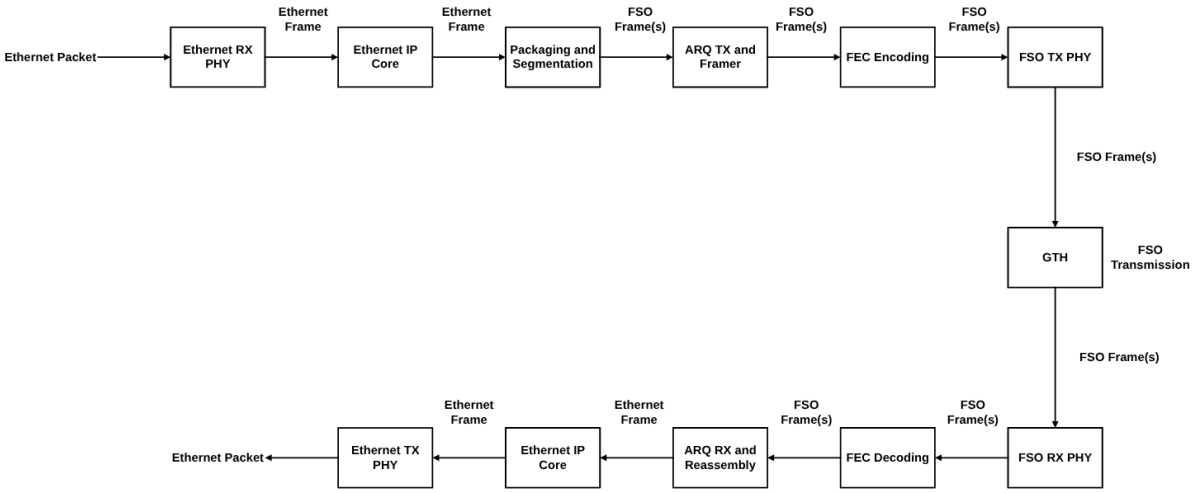


Figure 4. Ethernet Encapsulation

The encapsulation process begins when an Ethernet packet enters through the Ethernet RX PHY and is forwarded to the Ethernet IP Core. The data is then segmented into FSO frames by the Packaging and Segmentation block, followed by header addition in the ARQ TX and Framer, and error protection via FEC Encoding. After passing through the FSO TX PHY, the frames are transmitted over the air by the GTH transceiver as an FSO Transmission. On the receiving end, the GTH captures the signal, sending it through the FSO RX PHY to be corrected by FEC Decoding. The validated FSO frames are then reordered and stitched back together by the ARQ RX and Reassembly block into a complete Ethernet frame, which is finally routed out through the Ethernet IP Core and Ethernet TX PHY to emerge as the original Ethernet packet.

Following SDA OISL Standards on Page 42, Figure 2-16: Ethernet Encapsulation in 2.6.3.7 Ethernet Packet Encapsulation.

2.6.3.7 Ethernet Packet Encapsulation

2.6.3.7.1 Overview of Ethernet Packet Encapsulation

The OISL data plane encapsulates Ethernet packets as Free Space Optical (FSO) frames for transport across the optical link. The processing steps are illustrated functionally in Figure 2-16.

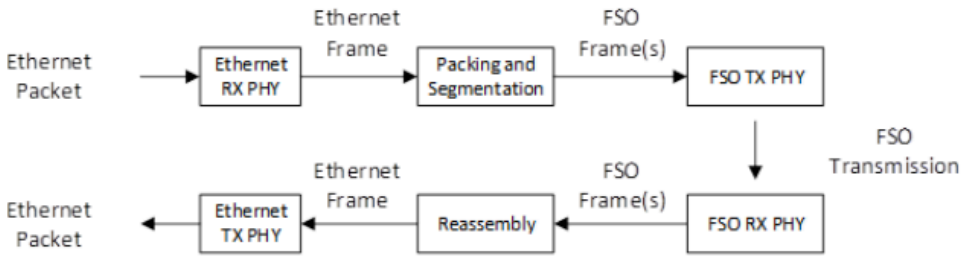


Figure 2-16: Ethernet Encapsulation

This scheme permits the OISL to carry virtually any form of Ethernet traffic transparently between a pair of optical terminals. The presence of the OISL is largely transparent to the endpoints.

Figure 5. SDA OISL Standard for Ethernet Packet Encapsulation

WiComAI Lab
Wireless Communications and Artificial Intelligence Lab

2. System Architecture Breakdown

2.1 Ethernet Interface TX

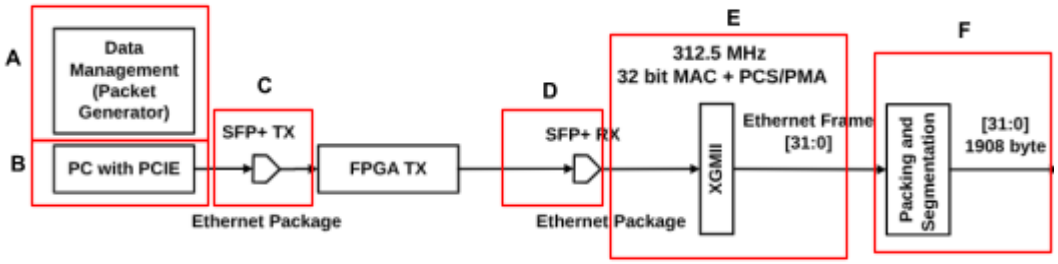


Figure 6. Ethernet Data Transmission

A. Data Management (Packet Generator)

The data management section (Packet Generator) is responsible for creating and organizing the data to be transmitted. In our simulation, it will send Ethernet packets based on the IEEE 802.3 standard Ethernet frame structure. These transmitted packets consist of a Destination MAC, Source MAC, EtherType/Length, Payload, and a Frame Check Sequence (FCS).

B. PC with PCIe

The PC on the transceiver side generates and transmits data through the PCIe interface. Our system uses the LANstar LS-PCIE-X520-DA2 Intel 82599ES PCIe Network Interface Card (NIC), as shown in Figures 1.a and 1.b. We chose this specific PCIe NIC because it features dual SFP+ ports capable of supporting data rates up to 10 Gbps.

C. SFP+ Tx

The SFP+ Tx module acts as a bridge between the electronic system and the optical communication medium. It receives high-speed electrical signals from the transmitter and converts them into optical signals to be sent to the FPGA module via a fiber optic cable. Using SFP+ modules allows for high-speed data transmission (10 Gigabit Ethernet).

D. SFP+ Rx

The SFP+ Rx module serves as the receiving interface in the optical communication system. It takes the optical signals sent from the transmitting PC over the fiber optic cable and converts them back into high-speed electrical signals. The FPGA transceiver then processes this data and forwards it to the XGMII interface for further processing.

E. XGMII Module Tx

XGMII serves as a high-speed data interface connecting the Ethernet system with the FPGA's data processing unit. In our design, data is processed over a 32-bit parallel bus, meaning 4 bytes of data are transferred simultaneously during every clock cycle. Operating at 312.5 MHz, this 32-bit configuration yields a theoretical bandwidth of 10 Gb/s (32 bits × 312.5 MHz). This setup allows the system to handle 10 Gigabit Ethernet speeds in parallel within the FPGA, as illustrated by the XGMII IP Core settings in Figure 7.

WiComAI Lab
Wireless Communications and Artificial Intelligence Lab

On the Ethernet side, the system follows the 10GBASE-R standard (IEEE 802.3). Before transmission, the Ethernet data is processed by the Physical Coding Sublayer (PCS) using 64b/66b encoding. Additionally, the FPGA transceiver uses a 156.25 MHz reference clock, matching the physical crystal oscillator standard on the FPGA board for 10G networks. Finally, before the data moves to the packing and segmentation stage, the XGMII module strips away the preamble and start frame delimiter (SFD) in accordance with sub-chapter 2.6.3.7.2 of the SDA OISL standard book.

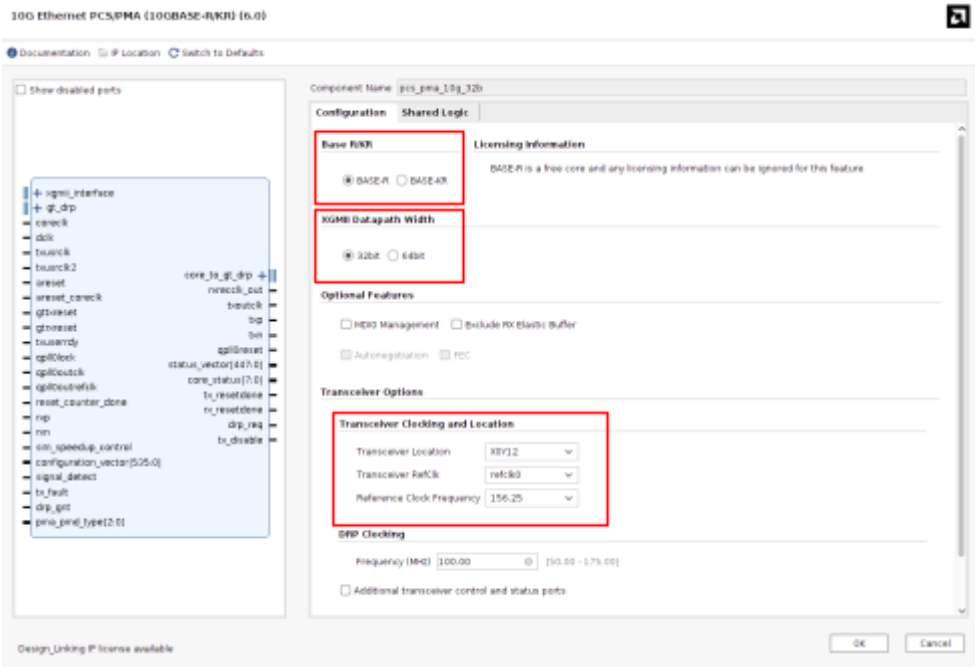


Figure 7. XGMII IP Core Setting

F. Packing and Segmentation

The Packing and Segmentation stage is a data processing step that occurs after the Ethernet frame is received. Its primary purpose is to organize the Ethernet data into a format compatible with the framing process detailed in sub-chapter 2.6.3.7.4 (FSO Frame Payload Format) of the SDA OISL standard book.

The first step is packing, which structures the data to match this specific frame format. The next step is segmentation. If the data exceeds the maximum FSO frame capacity of 1912 bytes, segmentation breaks it down into smaller, manageable pieces. Conversely, if the data is smaller than the standard FSO frame size, zero padding is added to reach the required length.

WiComAI Lab
Wireless Communications and Artificial Intelligence Lab

2.2 FSO Framer TX

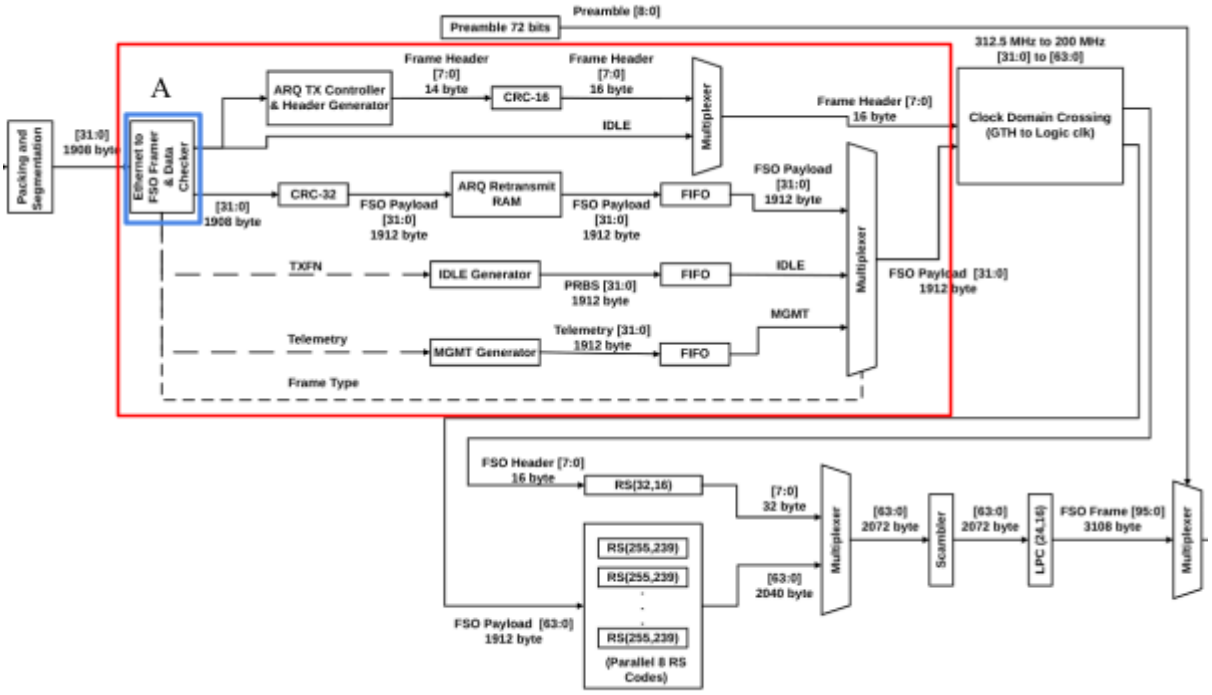


Figure 8. FSO Framer TX

The FSO Framer TX is responsible for converting packed and segmented Ethernet data into FSO frames that match the communication format of the SDA OISL system. At this stage, the Ethernet data isn't sent directly over the optical link. Instead, it is processed first to construct the payload, attach header information, verify data integrity, and determine the appropriate frame type for transmission.

Highlighted in the **blue box (A) in Figure 8**, the **Ethernet to FSO Framer & Data Checker** represents the very first step in constructing an FSO frame. Its primary job is to take Ethernet data that has already been through **packing and segmentation** and convert it into a payload format that the FSO Framer can actually use. In this specific architecture, the incoming Ethernet data is exactly **1908 bytes based on SDA OISL standard**, which directly becomes the **FSO Payload Data**. To make this happen, the block performs a few critical tasks seamlessly. It **receives the pre-processed data**, cross-checks it to ensure the **format and length** perfectly match the FSO frame standards, and then categorizes the incoming traffic—determining whether it is a standard **Ethernet payload, an idle packet, or a MGMT packet**. Once everything is verified and sorted, it carefully **prepares the payload data** so it is fully ready to be integrated into the final FSO frame structure.

WiComAI Lab
Wireless Communications and Artificial Intelligence Lab

2.2.1 ARQ Controller & Header Generator TX

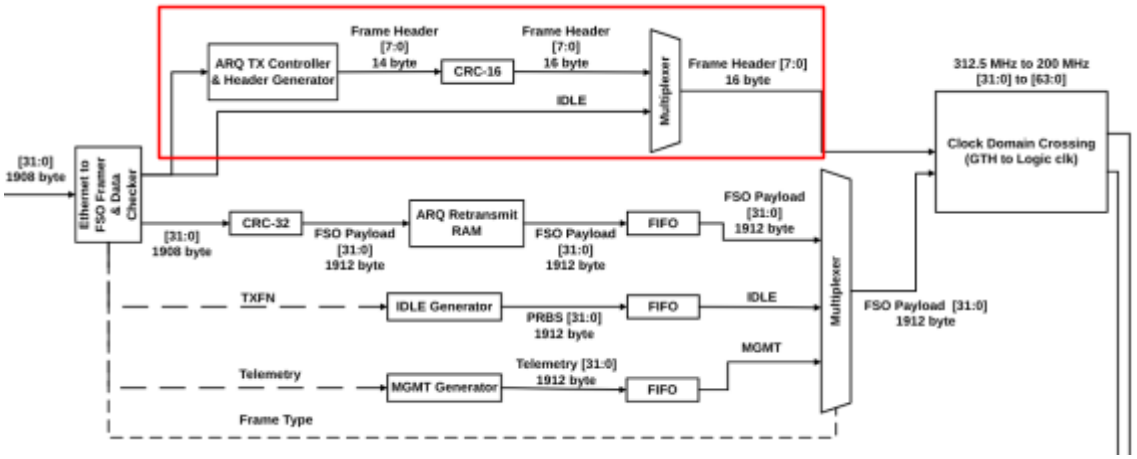


Figure 9. ARQ Controller & Header Generator TX

The **ARQ TX Controller & Header Generator** is essentially the brain behind building the FSO frame header and managing the control data needed for transmission and ARQ mechanisms. It constructs the header based strictly on the **Frame Header Fields** outlined in the SDA OISL standard (sub-section 2.6.3.3.1). In this specific architecture, the raw header initially starts at **14 bytes**. It then goes through a **CRC-16 calculation**, which brings the final frame header to exactly **16 bytes (128 bits)** to fully comply with the standard's requirements. On the operational side, the **ARQ TX Controller** acts as a safety net. It handles data retransmissions just in case the receiver catches a corrupted frame and the ARQ system requests a resend. Furthermore, whenever there isn't an active Ethernet payload ready to go, the system triggers an **IDLE line**. This cues the frame generation logic to start pumping out **IDLE frames** instead. This is a critical fallback feature because, in inter-satellite FSO communication, you cannot just leave the channel empty; the optical link requires a **continuous stream of data** at all times to keep the connection from dropping.

2.2.2 Payload Generator TX

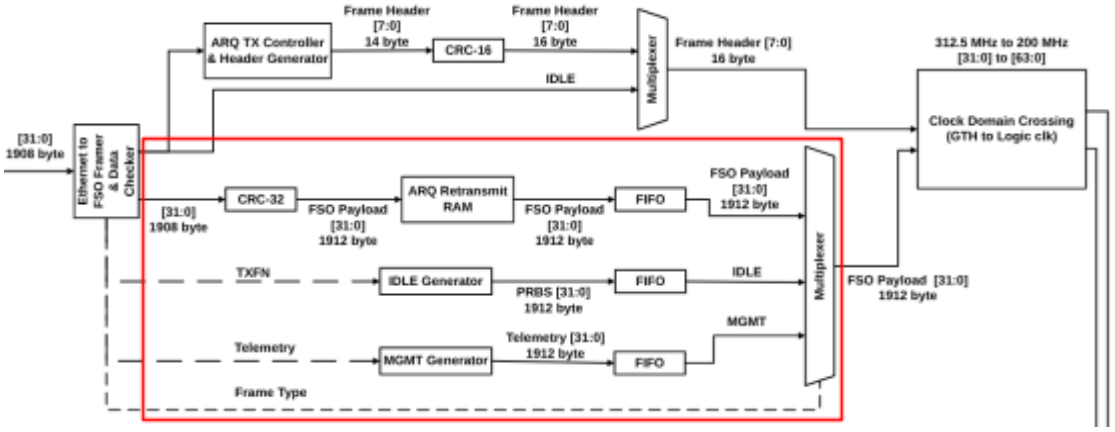


Figure 10. Payload Generator TX

The **Payload Generator TX** is responsible for preparing the payload portion of the FSO frame before it is forwarded to the Payload FEC stage. In the implemented architecture, the payload can originate from three different sources: **Ethernet DATA, IDLE, and MGMT/Telemetry**. These three frame types are multiplexed into a single FSO transmission stream according to the FRAME_TYPE mechanism defined in the SDA OISL architecture. It is shown in figure 11.

WiComAI Lab
Wireless Communications and Artificial Intelligence Lab

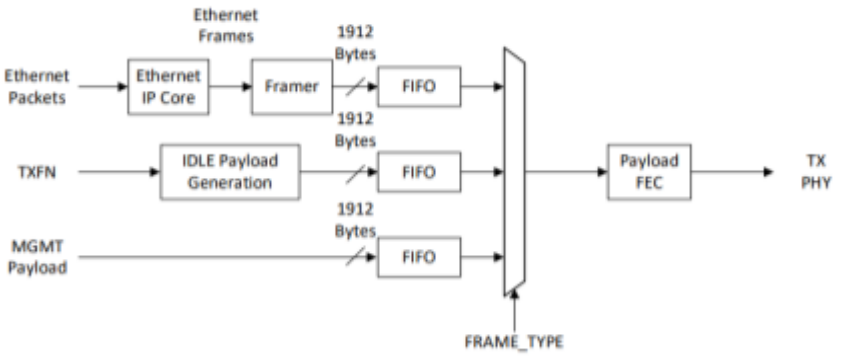


Figure 11. Multiplexing Frame Types

For the **Ethernet traffic**, the initial data sits at **1908 bytes**. The system then generates and **appends a CRC-32 value (adding 4 bytes)** to the very end based on **SDA OISL section 2.6.3.6.9**, bringing the total payload size to exactly 1912 bytes. This 1912-byte payload is safely tucked away in the **ARQ Retransmit RAM** just in case the ARQ mechanism requests a retransmission, before passing through a FIFO buffer to keep the processing flow smooth.

Whenever there's a lull in traffic—meaning no DATA or MGMT frames are ready to go—the **IDLE Generator** kicks in. It pumps out a PRBS-based IDLE payload through a FIFO buffer to ensure the **FSO optical link stays active with a continuous flow of frames**. Meanwhile, the **MGMT Generator** actively handles all the management and telemetry data, buffering it through its own FIFO.

The Payload Multiplexer steps in like a traffic controller. It selects which of the three payloads to send next based strictly on the **FRAME_TYPE tags: '01' for DATA, '00' for IDLE, and '10' for MGMT**. The final output is a clean, ready-to-go 1912-byte FSO Payload that moves directly to the Payload FEC. This clever setup allows regular Ethernet data, management info, and IDLE signals to seamlessly share the exact same FSO transmission link while strictly obeying the SDA OISL frame structure.

2.3 Clock Domain Crossing TX (GTH to Logic Clk)

In the proposed FPGA implementation, the generated header and payload data must be safely transferred between different processing clock domains through the Clock Domain Crossing (CDC) module. **It is important to note that while the SDA OISL standard dictates the framing and error-correction protocols, the CDC is a hardware-specific mechanism and an absolute technical necessity for 10 Gbps FPGA realization.**

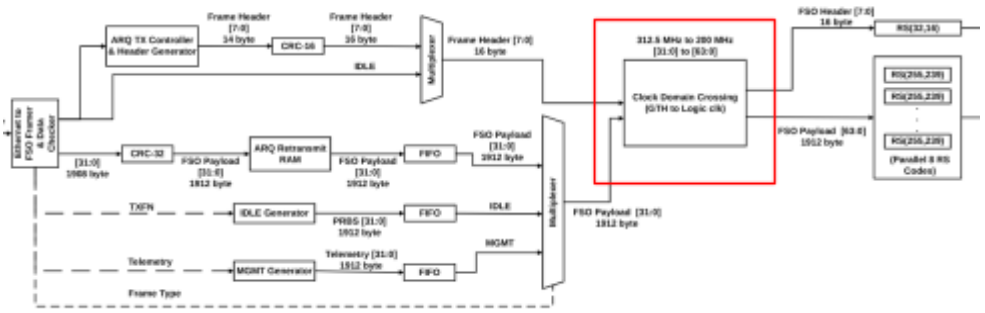


Figure 12. CDC for Clock Domain Crossing Frequency

WiComAI Lab
Wireless Communications and Artificial Intelligence Lab

The urgency of this module arises from the strict physical timing constraints of the hardware. The incoming Ethernet data operates on a high-speed, narrow bus (312.5 MHz at 32-bit) to sustain the massive 10 Gbps throughput. However, **the subsequent Reed-Solomon (FEC) processing stages require a wider data path** (e.g., transitioning to 64-bit processing) operating at a slower, stable frequency (such as 200 MHz) to execute complex parallel mathematics without causing internal logic timing violations.

To safely bridge these two asynchronous and incompatible clock domains, the CDC module heavily relies on a dual-clock FIFO (First-In, First-Out) buffer architecture. The FIFO acts as an elastic shock absorber; it securely queues the high-speed incoming data written by the 312.5 MHz clock and smoothly transitions it to be read by the slower 200 MHz logic domain. This crucial buffering mechanism ensures a continuous data flow, entirely preventing metastable states, data corruption, or dropped frames before the rigorous error-correction encoding begins.

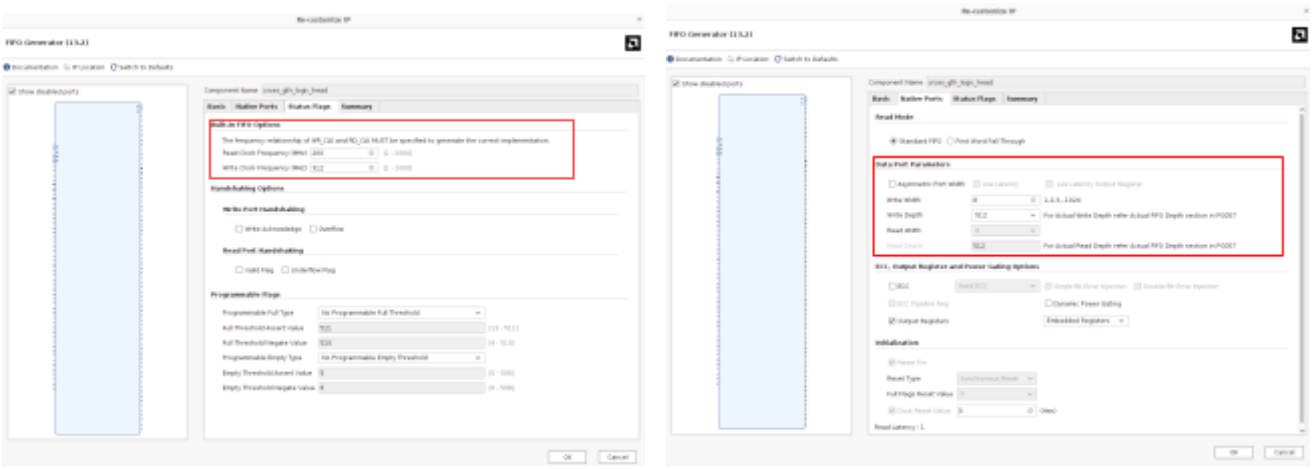


Figure 13. CDC for Data header

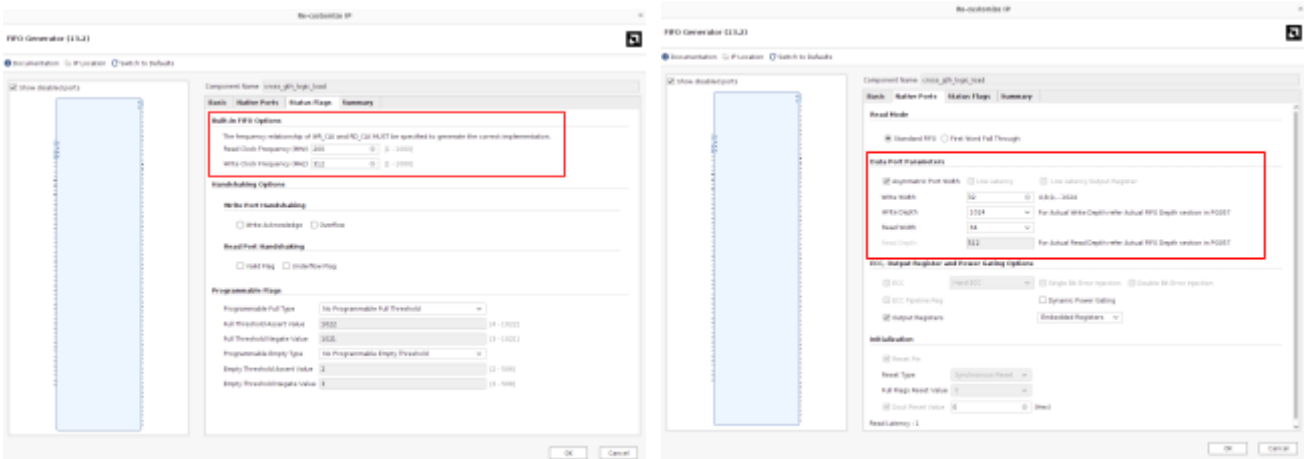


Figure 14. CDC for Data Payload

2.4 FEC Encoder TX

The Forward Error Correction (FEC) module provides robust error-detection and correction capabilities to mitigate signal degradation across the free-space optical channel. Following the **SDA OISL Standard, Section 2.6.3.6**, this stage divides the data processing into two parallel Reed-Solomon (RS) encoding paths:

WiComAI Lab
Wireless Communications and Artificial Intelligence Lab

- Header FEC: **As defined in SDA OISL Section 2.6.3.6.2**, the 16-byte header is processed through a shortened RS(32,16) encoder. This operation appends 16 parity bytes, producing a 32-byte highly protected header.
- Payload FEC: **As detailed in SDA OISL Section 2.6.3.6.3**, the 1912-byte payload is distributed across 8 parallel RS(255,239) encoders utilizing an interleaving depth of 8. This generates 128 parity bytes, expanding the payload to 2040 encoded bytes.

These two outputs are subsequently concatenated to form a single 2072-byte encoded data block, seamlessly transitioning to the scrambling stage.

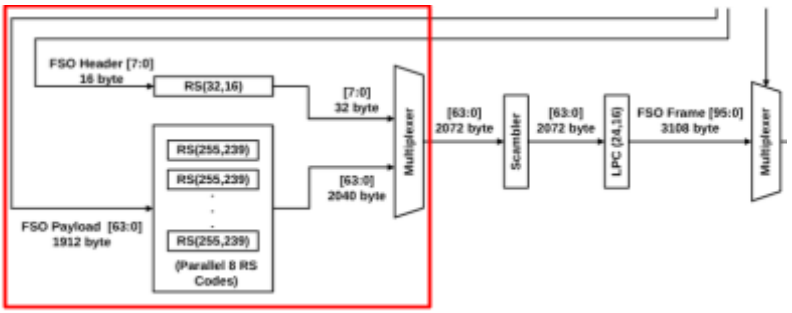


Figure 15. Architecture from Reed Solomon into GTH

2.5 Scrambler Encoder TX

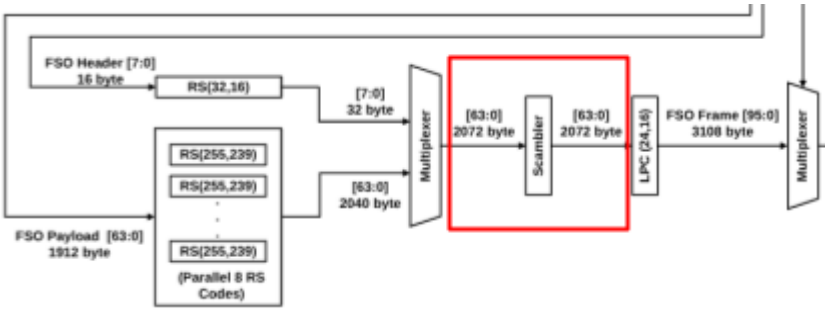


Figure 16. Scrambler Encoder TX

To maintain DC balance and prevent the transmission of long, repetitive bit patterns that could cause the receiver's Clock Data Recovery (CDR) to lose lock, the 2072-byte data block is heavily randomized. In strict accordance with the **SDA OISL Standard, Section 2.6.3.6.4 and Section 2.6.3.6.7,** the scrambler applies a Pseudo-Random Binary Sequence (PRBS).

As illustrated in **SDA OISL Figure 2-12 (Application of the Scrambler)**, the scrambling logic is applied continuously across the combined encoded header and payload. Crucially, the 72-bit modem-frame preamble completely bypasses this stage and remains unscrambled to serve as a recognizable synchronization marker. To ensure precise synchronization at the receiving node, the scrambler's state is strictly reset to a specific initialization pattern at the exact beginning of every modem frame header.

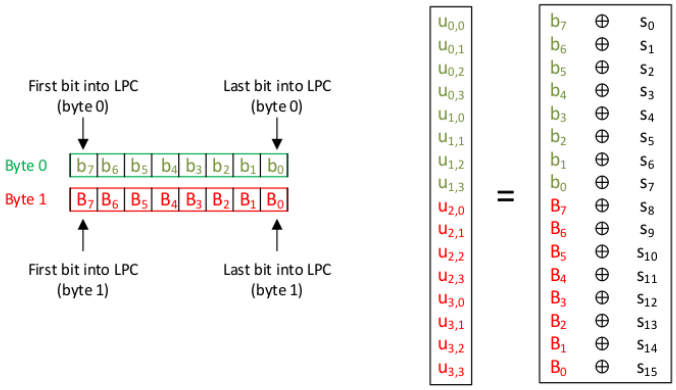


Figure 17. Application of the Scrambler

2.6 LPC Encoder TX

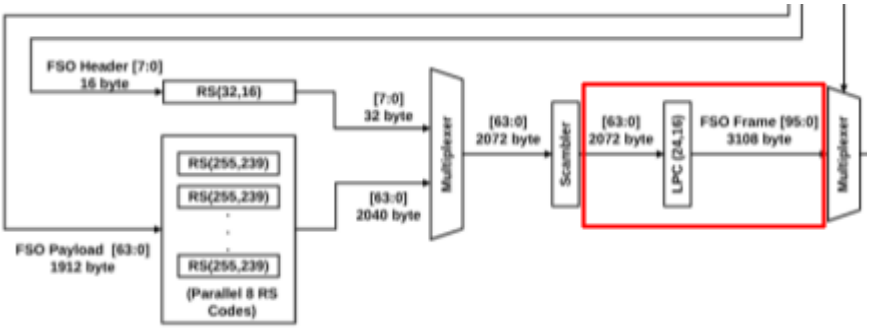


Figure 18. LPC Process

The Line Product Code (LPC) module performs the final channel-coding operation before multiplexing. **Based on the SDA OISL Standard, Section 2.6.3.6.8,** this block actively manages the running disparity of the optical signal, ensuring the optical transmitter operates safely within its designated envelope by minimizing the difference between transmitted logical ones and zeros.

The scrambled 2072-byte data block is processed through an LPC(24,16) encoder. As visually depicted in **SDA OISL Figure 2-13 (Differentially Encoding in LPC Encoder) and Figure 2-14 (LPC Codeword)**, the differential encoding mathematically expands the data by mapping every 16 bits of input into 24 bits of output. This specific expansion efficiently transforms the 2072-byte block into a fully formatted 3108-byte block, which is then ready to be multiplexed with the preamble for final optical transmission.

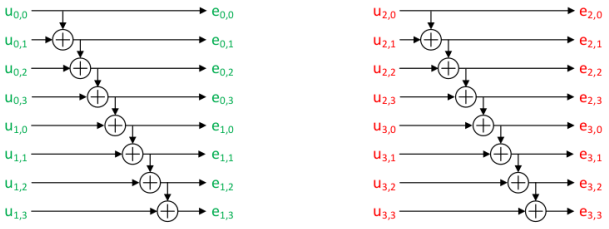


Figure 2-13: Differentially Encoding in LPC Encoder

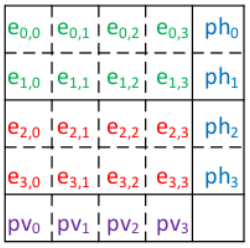


Figure 2-14: LPC Codeword

Figure 19. LPC Process

2.7 Multiplexer for FSO Framer Encoder TX

The Multiplexer acts as the final assembly point for the data before it leaves the internal framing logic. Its primary function is to construct the complete modem frame by prepending the

WiComAI Lab
Wireless Communications and Artificial Intelligence Lab

72-bit (9-byte) synchronization Preamble to the 3108-byte LPC-encoded data block (which contains the header and payload).

In strict accordance with the **SDA OISL Standard, Section 2.6.3.2**, the Preamble serves as a fixed synchronization marker for the receiver and must be transmitted in "plain text." Therefore, the Multiplexer ensures that the Preamble entirely bypasses both the Scrambler and the LPC(24,16) encoder. The successful multiplexing of these components results in a complete, ready-to-transmit 3117-byte FSO frame. At this stage in the FPGA architecture, the completed frame is formatted over a wide 128-bit data bus operating at 200 MHz.

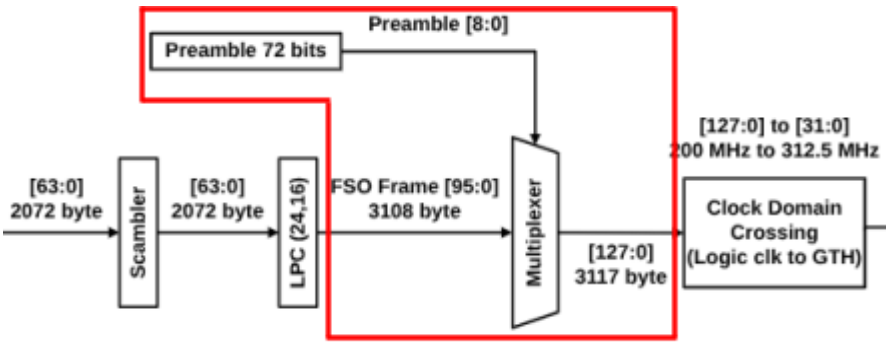


Figure 20. Multiplexer for FSO Framer Encoder TX

2.8 Clock Domain Crossing (Logic clk to GTH) Encoder TX

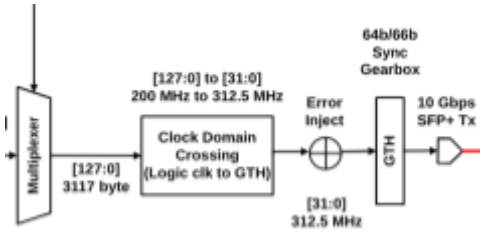


Figure 21. Cross Domain Crossing Logic to GTH

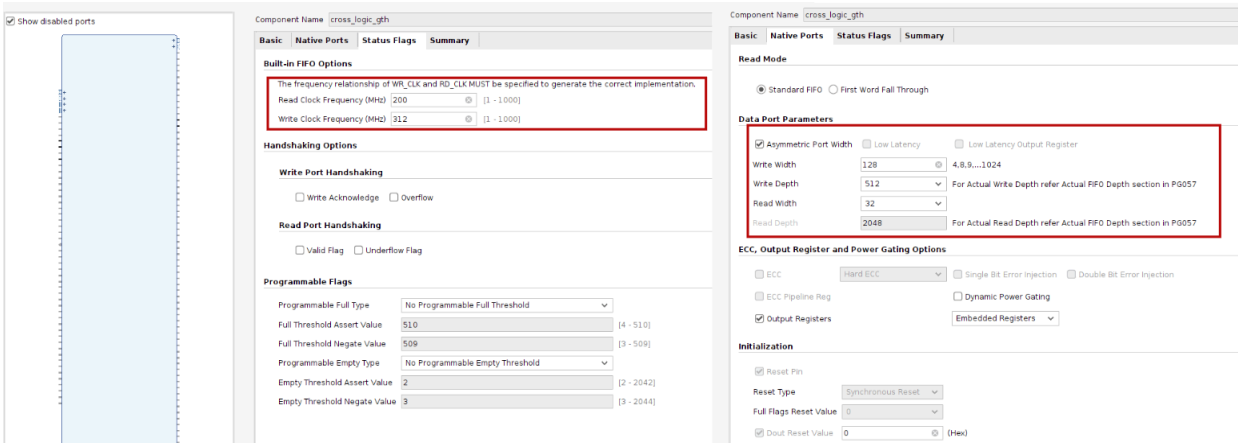


Figure 22. IP for Logic to GTH

Before the fully constructed 3117-byte frame can be transmitted, it must pass through the Clock Domain Crossing (CDC) module. **While CDC is a hardware-specific implementation rather than a protocol defined by the SDA OISL standard, it is an absolute technical necessity to achieve a 10 Gbps link on an FPGA.** The urgency of this module stems from the physical timing limitations

WiComAI Lab
Wireless Communications and Artificial Intelligence Lab

of hardware logic. The heavy mathematical operations required by the Reed-Solomon and LPC encoders demand a wide 128-bit parallel bus operating at a stable, lower frequency of 200 MHz to prevent internal logic timing violations. **However, the Gigabit Transceiver (GTH) hardware strictly requires a narrow 32-bit data bus operating at exactly 312.5 MHz to physically push data at a 10 Gbps line rate.** The CDC module acts as the critical asynchronous bridge between these two incompatible environments. It safely converts the 128-bit/200 MHz processed data into the required 32-bit/312.5 MHz format without introducing metastable errors or data loss. Without this CDC module, it is physically impossible to synchronize the heavy baseband processing with the ultra-high-speed optical transmitter.

2.9 GTH

The Gigabit Transceiver (GTH) serves as the Physical Medium Attachment (PMA) and Physical Coding Sublayer (PCS), acting as the critical bridge between the FPGA's internal parallel logic and the ultra-high-speed serial optical interface. Configured via the AMD UltraScale FPGAs Transceivers Wizard, the GTH ensures seamless 10 Gbps transmission strictly compliant with the **IEEE 802.3 10GBASE-R Ethernet standard**.

The GTH operates through two distinct pipelines:

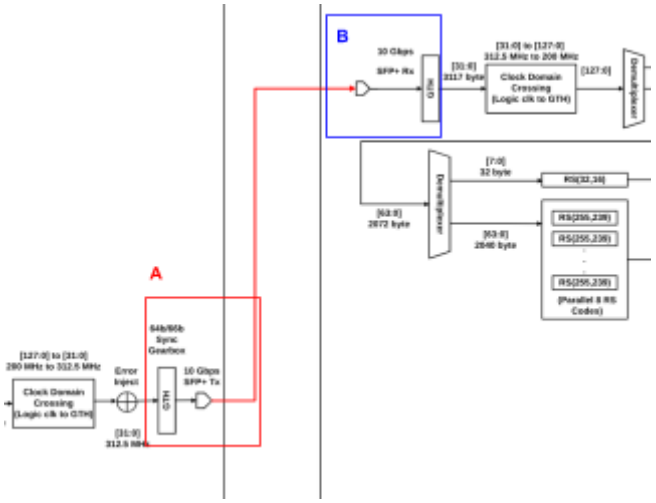


Figure 23. GTH Transferring use 64b/66b

A. GTH TX (Transmission Pipeline)

Before the data leaves the FPGA, the GTH TX prepares the 32-bit parallel data (operating at 312.5 MHz) for serial transmission.

- **64b/66b Sync Gearbox:** In strict accordance with the IEEE 802.3 standard for 10 Gigabit Ethernet, This module formats the data by adding a 2-bit synchronization header to every 64-bit data block. To safely insert this header without losing data, the gearbox periodically pauses the data flow for 1 cycle every 31 cycles.
- **Bit-Reversal and Serialization:** The GTH automatically performs a bit-reversal operation to match the transmission bit-order (right-most bit/LSB transmitted first) required by the protocol. **The Parallel-In-Serial-Out (PISO)** block then converts the aligned parallel data into a continuous 10 Gbps serial electrical stream to drive the external SFP+ TX module.

B. GTH RX (Reception Pipeline)

Upon receiving the 10 Gbps optical signal from the SFP+ RX module, the GTH RX reverses the transmission process to safely extract the data.

WiComAI Lab
Wireless Communications and Artificial Intelligence Lab

- **Clock Data Recovery (CDR) & Deserialization:** The incoming high-speed serial stream is first captured by the RX Analog Front End and CDR circuits, which extract the embedded clock and convert the stream back into parallel data via the **Serial-In-Parallel-Out (SIPO)** block.
- **Gearbox Slip and Block Alignment:** Because the serial bits arrive continuously at extreme speeds, the hardware must actively locate the exact packet boundary. The RX Sync Gearbox continuously evaluates the stream for the valid **IEEE 802.3 64b/66b synchronization headers (01 or 10)**. If the alignment is incorrect, the module triggers a gearbox slip operation, which incrementally shifts the internal read pointer by 1 bit until exact block alignment and synchronization lock are achieved.
- **Bit-Reversal Restoration:** Once locked, the system reverses the bit order back to its original internal format. The aligned and synchronized 32-bit data is then forwarded to the Clock Domain Crossing block to begin the frame decoding process.

2.10 Demultiplexer for Decoder RX

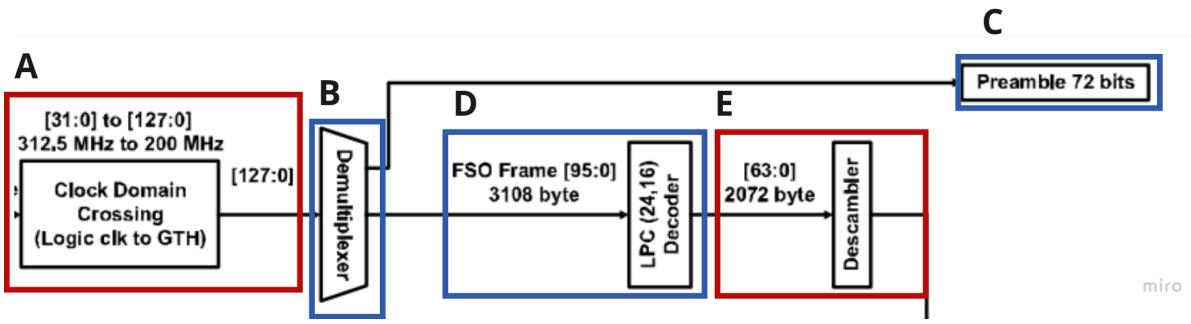


Figure 24. Demultiplexer for Decoder Processing

A. The Clock Domain Crossing converts the received data from **32-bit at 312.5 MHz to 128-bit at 200 MHz.** This conversion is implemented to match the data width and clock required by the following processing stage.

B. The Demultiplexer separates the received data into the required sections for further processing. **Based on the SDA OISL standard**, the frame contains a **72-bit preamble followed by the header and payload**.

C. The **72-bit preamble** is separated from the main frame for synchronization purposes. **Based on the SDA OISL standard**, the preamble is transmitted before the header and payload and is not processed by LPC or scrambling.

D. The data is processed by the **LPC(24,16) Decoder** to recover the data from the LPC encoding process. **Based on the SDA OISL standard, Section 2.6.3.6.8**, LPC(24,16) is applied to the header and payload.

E. The data is then processed by the Descrambler to recover the original data from the scrambling process. **Based on the SDA OISL** standard, the header and payload are scrambled after Reed-Solomon encoding, so the receiver descrambles the data before Reed-Solomon decoding.

WiComAI Lab
Wireless Communications and Artificial Intelligence Lab

2.11 Demultiplexer for FEC Decoder RX

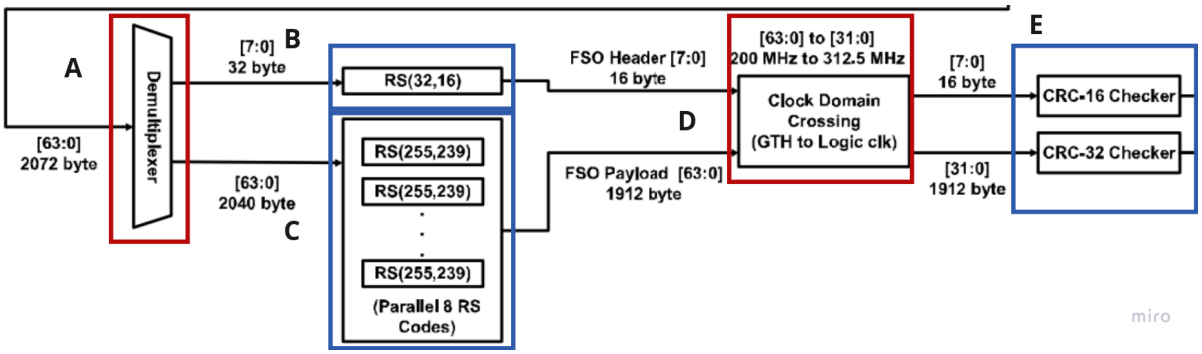


Figure 25. Demultiplexer for Reed Solomon Processing in Decoder

A. The Demultiplexer separates the decoded data into the **FSO Header and FSO Payload** so that each part can be processed using the appropriate error-correction scheme.
B. The FSO Header is processed using an **RS(32,16) Decoder** to correct errors in the received header. **Based on the SDA OISL standard, Section 2.6.3.6.2**, the header uses the RS(32,16) code.
C. The FSO Payload is processed using **eight RS(255,239) Decoders in parallel**. **Based on the SDA OISL standard, Section 2.6.3.6.3**, the payload uses eight RS(255,239) codes with an interleaving depth of eight.
D. The Clock Domain Crossing converts the data from **64-bit at 200 MHz to 32-bit at 312.5 MHz**. This conversion is implemented to match the data width and clock required by the following interface and to support the **10 Gbps line rate**.
E. The CRC Checker verifies the integrity of the received data. Based on the **SDA OISL standard, Section 2.6.3.6.1**, CRC-16 is used for the header and CRC-32 is used for the payload. If an error is detected in the header. Based on the **SDA OISL Standard, section 2.6.3.3.1,** the ARQ process is triggered and a NAK is generated, causing the system to return to the ARQ process. If the header is valid, an ACK is generated and the data proceeds to the next processing stage.

2.12 Reassembly for RX

The Reassembly and ARQ RX Controller module is the final critical processing stage within the **FPGA RX architecture (Figure 3).** Its primary responsibility is to validate incoming frames, execute the receiver-side ARQ logic, and reconstruct the fragmented payloads into standard network packets. This process directly correlates with the Orange Path (RX Node) illustrated in the ARQ operational flow **Figure 28. ARQ System Architecture**.

WiComAI Lab
Wireless Communications and Artificial Intelligence Lab

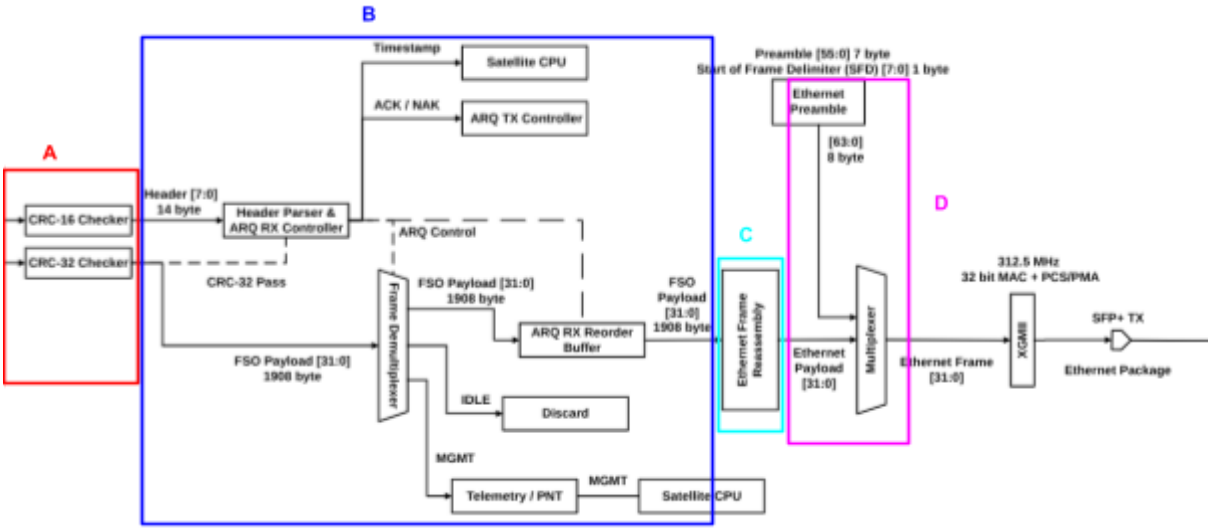


Figure 26. Reassembly and ARQ RX Controller

A. Data Integrity Check (CRC Evaluation)

The reassembly process commences by verifying the integrity of the incoming data stream to ensure no corruption occurred across the atmospheric channel. Following the **SDA OISL Standard, Section 2.6.3.6.1**, the CRC-16 Checker module validates the 14-byte Header, while the CRC-32 Checker evaluates the 1908-byte Payload. The success of the CRC-32 check is the absolute prerequisite for the ARQ system, corresponding to the "CRC-32 Passed?" decision block in the **ARQ Flowchart Figure 28. ARQ System Architecture in this document**. **According to the SDA OISL section 2.6.3.3.3**, a data packet is only considered successfully received and eligible for a positive confirmation signal if it successfully passes this CRC-32 check.

B. Header Parsing and ARQ Routing

Once validated, the Header Parser & ARQ RX Controller interprets the frame label to route the data content appropriately. Empty data (IDLE) is discarded, while management telemetry (MGMT) is routed to the Satellite CPU. For data payloads, the controller executes the Selective Repeat protocol specifically defined in the **SDA OCT Standard Version 4.0.0, Section 3.4.6.1.2.** If the CRC-32 fails, the controller generates a Negative Acknowledgement (NAK) and drops the corrupted frame, as illustrated in the failure path of Figure 28. ARQ System Architecture. Conversely, valid frames trigger a positive acknowledgment (ACK), utilizing the updated ACK_START_FN and ACK_SPAN fields to efficiently confirm reception back to the transmitting node.

C. Packet Reordering and Reassembly

Valid communication payloads are temporarily routed into the ARQ RX Reorder Buffer (which interfaces with external DDR4 memory to handle the massive storage required by the 35 ms Round-Trip Time). As detailed in the **SDA OISL Standard, Section 2.6.3.7.7**, this queue space buffers incoming packets until any dropped frames are successfully retransmitted and acknowledged. Once the packets are accurately and sequentially ordered—matching the "Place frames into correct sequential order" step in **Figure 28. ARQ System Architecture**—the data fragments are extracted and stitched back together by the Ethernet Frame Reassembly block into a complete, dynamically sized frame that complies with the **standard IEEE 802.3 format**.

WiComAI Lab
Wireless Communications and Artificial Intelligence Lab

D. Preamble Multiplexing and Output

The fully assembled and sequentially ordered data packet is then processed by the final Multiplexer module. In accordance with the **physical specifications of the IEEE 802.3 Standard**, the packet is prepended with standard synchronization markers—specifically, a 7-byte Ethernet Preamble and a 1-byte Start of Frame Delimiter (SFD). This final combination process converts the reconstructed payload back into an original, recognizable Ethernet packet, which is then routed through the XGMII interface to the outbound SFP+ TX port for the Host PC, successfully completing the reception cycle.

2.13 Ethernet Interface RX

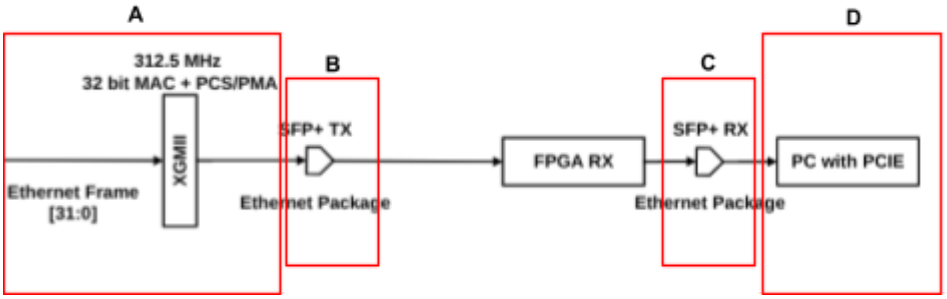


Figure 27. Ethernet Data Reception

A. XGMII Module Rx

The XGMII Rx Module serves as a bridge between the decoder output on the Rx FPGA and the Network Interface Card (NIC) on the receiving PC. It acts as the final translator, converting the FPGA's internal data stream back into the standard 10 Gbps Ethernet physical format for the PC receiver. This process ensures that the receiving PC's NIC can recognize and read the complete Ethernet frames without flagging them as damaged or corrupted packets. On the Rx side, XGMII IP Core settings are the same as XGMII IP Core settings on the Tx side.

B. SFP+ Tx

SFP+ Tx module on the decoder side acts as a link between the electronic system and the optical communication medium. It receives high-speed electrical signals from the decoder and converts them into optical signals. These signals are then transmitted over a fiber optic cable to the receiving PC. Using the SFP+ module enables high-speed data transmission at 10 Gigabit Ethernet speeds.

C. SFP+ Rx

On the receiving PC, the SFP+ Rx module serves as the receiving interface for the optical communication system. It takes the optical signals sent over the fiber optic cable from the FPGA Rx's SFP+ module and converts them back into high-speed electrical signals, which are then finally received and processed by the PC.

D. PC with PCIe

As the counterpart to the transceiver side, the PC on the receiver end is responsible for taking in data via the PCIe interface. Our system utilizes the LANstar LS-PCIE-X520-DA2 Intel 82599ES PCIe Network Interface Card (NIC), as illustrated in Figures 1.a and 1.b. We chose this specific PCIe NIC because it features dual SFP+ ports capable of supporting data transfer rates up to 10 Gbps.

WiComAI Lab
Wireless Communications and Artificial Intelligence Lab

3. ARQ Selective Repeat Architecture Breakdown

The Automatic Repeat Request (ARQ) mechanism is a critical data link layer function designed to guarantee lossless data transmission over the free-space optical medium. The system utilizes the **Selective Repeat** ARQ protocol as defined in the **SDA OCT Standard Version 4.0.0, Section 3.4.6.1.2**. This ensures that only specifically corrupted frames are retransmitted, optimizing bandwidth efficiency compared to traditional Go-Back-N methods.

3.1 Theoretical Capacity and Memory Calculation

To maintain a continuous 10 Gbps transmission rate without stalling (Stop-and-Wait), the TX ARQ Retransmit RAM must be capable of buffering all FSO frames currently "in flight" until their respective ACK or NAK responses are received from the far-end terminal.

For a target satellite **Line-of-Sight (LoS) distance of 5,000 km,** the physical memory requirement is dictated by the speed of light and the frame duration:

- **Propagation Delay (1-Way):**

$$\begin{aligned} Propagation\ Delay &= Target\ distance\ /\ Speed\ of\ light\ in\ vacuum \\ &= 5,000\ km\ /\ 300,000\ km/s. \\ &= 16.67\ ms \end{aligned}$$

- **Round-Trip Time (RTT):**

$$\begin{aligned} Round\ Trip\ Time\ (RTT) &= Propagation\ Delay\ x\ 2 \\ &= 16.67\ ms\ \times\ 2 \\ &\approx 33.34\ ms \end{aligned}$$

  Accounting for internal FPGA processing latency ($1.66\ ms$), the total turnaround time is rounded to **35 ms**.

- **Frame Duration:** According to SDA OISL Table 2-4 and SDA OCT V4.0.0 standard (Table 3-20), at a 10 Gbps line rate, a single FSO frame is transmitted in $2.494\ \mu s$.

- **Frames in Flight:**

$$\begin{aligned} Frames\ in\ Flight &= RTT\ /\ Frame\ Duration \\ &= 35\ ms\ /\ 2.494\ \mu s \\ &\approx 14,033\ frames \end{aligned}$$

Since each frame payload contains 1912 bytes, the **minimum required buffer size** is:

- **Required RAM Capacity:**

$$\begin{aligned} Required\ RAM\ Capacity &= Frames\ in\ Flight\ \times\ Size\ Payload \\ &= 14,033\ frames\ \times\ 1912\ bytes \\ &\approx 26.8\ Megabytes\ (MB) \text{ (or roughly 214.4 Megabits).} \end{aligned}$$

3.2 ARQ Protocol Processing Flow

The operational flow of the ARQ system is governed by the state parameters defined in the SDA OCT V4.0.0 Header (Table 3-8). As illustrated in the flowchart, the process is divided into Transmitter (TX Node, Blue blocks) and Receiver (RX Node, Orange blocks) operations. The operational flow of the ARQ Selective Repeat system is intricately tied to the specific hardware modules within our FPGA architecture (as detailed in Figures 2 and 3). The process is logically divided into four main stages, seamlessly integrating the baseband logic with the external DDR4 memory interface.

WiComAI Lab
Wireless Communications and Artificial Intelligence Lab

The Selective Repeat mechanism is executed through the following structured stages:

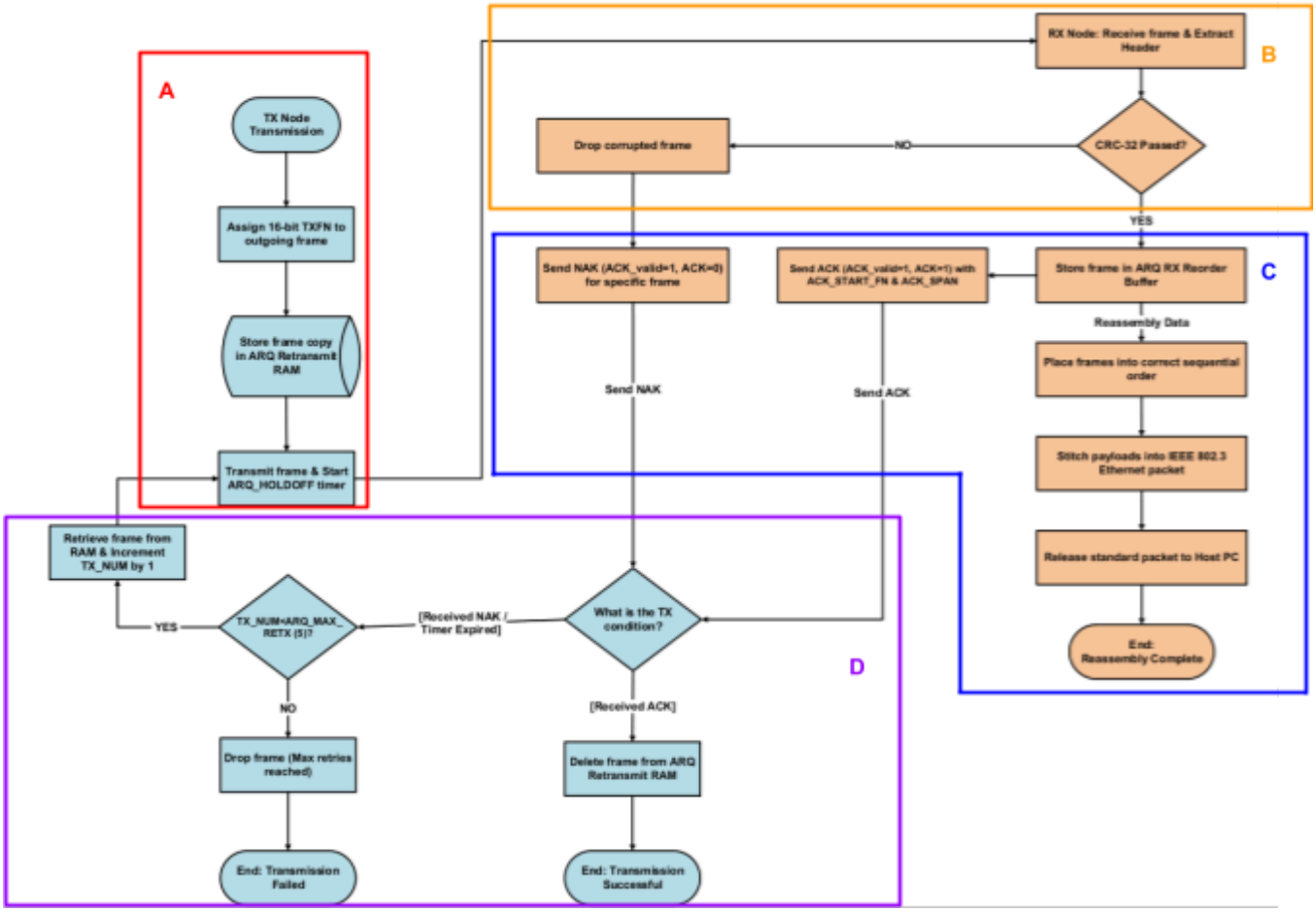


Figure 28. ARQ System Architecture

A. TX Node Initial Transmission & Storage (Blue Path)

The transmission cycle begins in the **TX Node (Figure 2)** when the Ethernet to FSO Framer & Data Checker block identifies a valid packet and routes it into the FSO Payload [31:0] 1912 byte pipeline. First, the ARQ TX Controller & Header Generator assigns a unique, sequential 16-bit TXFN (Transmit Frame Number) to the newly formed header, complying **with the base SDA OISL framing rules**. Concurrently, an exact copy of the 1912-byte payload is securely written into the external ARQ Retransmit RAM. Once the frame is multiplexed, encoded by the FEC blocks, and transmitted through the GTH optical interface, the ARQ TX Controller simultaneously initiates the ARQ_HOLDOFF timer (determined by the ARQ_HOLDOFF_NFRAMES parameter) to track the expected acknowledgment deadline.

B. RX Node Reception & Evaluation (Orange Path)

Upon traversing the optical link, the data enters the **RX Node (Figure 3)** through the GTH block and is decoded back into the header and payload paths. The Header Parser & ARQ RX Controller extracts the 14-byte header, while the 1912-byte payload is rigorously evaluated by the CRC-32 Checker in accordance with the SDA OISL error control standard. If the payload fails the CRC-32 check (the "NO" path in the flowchart), the frame is immediately dropped by the Frame Demultiplexer. The ARQ RX Controller then generates a Negative Acknowledgment (NAK) by setting ACK_valid = 1 and ACK = 0 for that specific frame, which is sent back to the TX Node. Conversely, if the CRC-32 Pass signal is asserted (the "YES" path), the valid payload is successfully routed into the ARQ RX Reorder Buffer to await further processing.

WiComAI Lab
Wireless Communications and Artificial Intelligence Lab

C. RX Node Acknowledgment & Reassembly (Orange Path)

Following a successful CRC-32 evaluation, the **RX Node (Figure 3)** executes two simultaneous operations. First, the ARQ RX Controller prepares a positive acknowledgment by setting ACK_valid = 1 and ACK = 1. Leveraging the advanced efficiency updates of the **SDA OCT V4.0.0 standard**, it utilizes the ACK_START_FN and ACK_SPAN fields to confirm multiple consecutive successful frames in a single reply. Concurrently, inside the ARQ RX Reorder Buffer, out-of-order frames are held until any missing frames successfully arrive. Once a contiguous sequence is formed, the payloads are passed to the Ethernet Frame Reassembly block, which stitches the data together, prepends the Ethernet Preamble, and routes the complete standard Ethernet Frame [31:0] through the XGMII interface to the Host PC.

D. TX Node Condition & Resolution (Blue Path)

Back at the **TX Node (Figure 2)**, the ARQ TX Controller continuously evaluates the incoming feedback loop from the remote terminal to determine the final transmission condition. If a valid ACK is received, the controller issues a command to permanently delete the corresponding payload copy from the ARQ Retransmit RAM, resulting in a successful transmission state. However, if a NAK is received, or if the ARQ_HOLDOFF timer expires, a retransmission is triggered. The controller verifies if the TX_NUM (transmission attempt counter) is strictly less than the ARQ_MAX_RETX limit **(configured up to 5 attempts per SDA OCT V4.0.0)**. If attempts remain, the controller retrieves the specific dropped FSO Payload directly from the ARQ Retransmit RAM, increments TX_NUM by 1, and injects the payload back into the Multiplexer for retransmission. If the maximum retry limit is reached, the frame is permanently dropped, resulting in a failed transmission state.

3.3 Hardware Implementation (External DDR4 Integration)

The calculated 26.8 MB memory requirement fundamentally **exceeds the internal Block RAM (BRAM) capacity of the ALINX AXKU062 Kintex UltraScale FPGA**, which is limited to approximately 4.7 MB (38.0 Mb).

To resolve this hardware limitation, the ARQ architecture is explicitly designed to interface with external memory:

1. **AXI4 Memory Interface Generator (MIG):** The ARQ Retransmit RAM (TX) and the ARQ RX Reorder Buffer (RX) are connected via AXI4 master modules to the external DDR4 SDRAM installed on the ALINX board.
2. **TX Flow:** As frames are generated, a copy of the 1912-byte payload is written into the DDR4 memory, indexed by its unique 16-bit TXFN. If an ACK is received, the memory sector is freed. If a NAK is received, the ARQ Controller retrieves the specific frame from the DDR4 and injects it back into the transmission multiplexer.
3. **RX Flow:** The ARQ RX Reorder Buffer temporarily stores out-of-order frames in the receiver's DDR4 memory. Once the missing (retransmitted) frames arrive and pass the CRC-32 check, the Reassembly module retrieves the strictly sequential payloads and stitches them back into **standard IEEE 802.3 Ethernet packets for the PC.**

WiComAI Lab
Wireless Communications and Artificial Intelligence Lab